



INFORMATICS
INSTITUTE OF
TECHNOLOGY

UNIVERSITY OF
WESTMINSTER 

Informatics Institute of Technology (IIT) Sri Lanka, Affiliated
with The University of Westminster, UK Module

Software Engineering Principles and Practice Coursework
5SENG007W

Hotel Booking System

Student Name: Jinudini Herath

UOW Number: w2149665

IIT Number: 20240384

Group Number: SE – Group 02

I. Abstract

The report contains the design of a web-based Hotel Booking System that is designed to enhance the efficiency of hotel bookings and management. The system enables guests to search, book hotels and rooms, and make secure online payments and hotel staffs to handle booking and room availability. A software development approach is Agile, and it is used to enhance flexibility and improvement in the short term. There is a three-tier architecture design in the system, which achieves separation of concerns, scalability, and maintainability. The UML diagrams are models based on the structure and behavior of the system such as class and sequence diagrams. Also, the report addresses the choice of the modern development tools and the use of the main principles of software engineering to enable the creation of a stable and scalable system design.

II. Acknowledgement

I would like to say that I am deeply grateful to the module lecturer and tutors who guided and supported me in this coursework. Their comments and the explanations made me have more insight into the software architecture and system design notions. I would also not want to forget to mention my peers whose discussions and support enabled me to complete this report successfully.

III. Table of Contents

I. Abstract.....	i
II. Acknowledgement.....	ii
III. Table of Contents	iii
IV. List of Figures	v
V. List of Tables	v
1. Project Background	1
2. Reflection on Software Engineering Methodology and Lifecycle	2
2.1 Selected Software Engineering Methodology.....	2
2.2 Reflection on the Software Development Lifecycle	3
3. System Architecture	4
3.1 Architectural Style.....	5
3.2 Architectural Tiers, Platforms, and Components.....	5
3.3 Connectors	7
3.4 Communication Types.....	7
4. Testing of your Architectural Design	8
4.1 Realization of Functional Requirements (FRs)	8
4.2 Realization of Non-Functional Requirements (NFRs)	9
4.3 Realization of Technical Requirements (TRs)	9
5. Class Diagram and Justification.....	10
5.1 Class Structure	10
5.2 Identification of Attributes and Actions	10
5.3 Justification for Classes, Attributes, and Actions	10
5.4 Correct UML Notation.....	11
6. Class Design Testing.....	13
6.1 Selected Core Functionality.....	13

6.2 UML Sequence Diagram Description	13
6.3 Required Classes and Their Roles.....	14
7. Considerations of Modern Tools	16
7.1 Development and Testing Tools	16
7.2 Implementation Tools	17
8. Considerations of the Software Engineering principles in your design.....	18
8.1 Separation of Concerns	18
8.2 Modularity	18
8.3 Abstraction	19
8.4 Anticipation of Change	19
8.5 Incremental Development	19
References	20
Appendix.....	21

IV. List of Figures

Figure 1: Architecture Diagram	4
Figure 2: Class Diagram	12
Figure 3: Sequence Diagram	15

V. List of Tables

Table 1: Connectors	7
---------------------------	---

1. Project Background

The client is the owner of a medium-sized hotel which offers accommodation services to local and international clients. Currently the hotel handles the reservations via various media such as making or placing them via phone calls, emails, walk-ins and third-party booking sites. Incidents like room assignment, guest data and reservation confirmation, and tracking of payment are done manually by use of spreadsheets and paper documents.

The hotel is often faced with overbooking problems, lack of consistent information regarding room availability and also delays in confirmation of booking, due to lack of a centralized system. Front-desk employees take their fair share of time updating the records and verifying the room availability in various channels, which makes it more probable that human error could occur and less efficient operations are performed. The data about the guests and records of payments is distributed among various sources where the retrieval process of the data takes time and is not reliable to retrieve the data.

The client will use a Hotel Booking System to overcome these challenges as it will automate the reservation processes and become a single platform where all the activities related to booking are integrated. The suggested system will facilitate the ability of the guests to search and book available rooms online, and complete secure payments using the web-based interface. The administrative dashboard will be given to hotel staff members to handle bookings, room status, check-in and check-out operations and hotel management will be enabled to view real-time occupancy, revenue and performance reports to aid in making informed decisions.

The project background and context are anchored on discussions held with the client through seminar sessions and on which the basis of the system design is set out in this report.

2. Reflection on Software Engineering Methodology and Lifecycle

2.1 Selected Software Engineering Methodology

Agile Scrum approach has been used in the development of the Hotel Booking System because of the dynamic and customer-based nature of the hotel business. Hotel guests, front-desk employees, and management are all the stakeholders of the project whose needs are bound to change over a period. As an example, price shifts, booking approaches, seasonal promotions, or the adoption of new forms of payment can be introduced in the development.

Agile encourages the ability to be flexible and receive feedback on a regular basis by its stakeholders, allowing the system to be created in steps and enhanced through recurring reviews. This is especially significant with the Hotel Booking System since initial versions of the features like room search, managing bookings, and processing payments can be tested by the hotel employees and enhanced prior to its full implementation.

The Agile platform alleviates the risk of poised design alterations and expensive reworking in opposition to the conventional approaches like the Waterfall one. Waterfall involves set requirements during the initiation of the project and this is inappropriate in a system that has to evolve with the needs of the business. Agile is also more beneficial than the Spiral model since it is less cumbersome and lighter in nature and does not entail the cumbersome risk analysis documentation as it does not involve a lot of risk analysis documentation as Spiral model does.

2.2 Reflection on the Software Development Lifecycle

The Agile lifecycle that is used in this project is an iterative and incremental model with the development of the system taking place in short development cycles called sprints. Every sprint is made up of requirement refinement, design, implementation, testing and review.

In the case of Hotel Booking System, the lifecycle starts with the high-priority features like room availability management and online booking. These are the basic functionalities and they are created and developed initially and then new features are added like the secure payment processing, administrative dashboards and reporting modules in the subsequent iterations. This strategy will make sure that the most important business functions are being provided as early as possible.

Design decisions can be checked continuously and at the end of every sprint, to make sure they hold in actual business conditions, thus cutting down defects and enhancing the quality of the system. Stakeholder feedback will be implemented in subsequent iterations and in this way will keep the system on track with the business goals. By and large, Agile lifecycle supports the maintainability, scalability and incremental improvement, and this tool is appropriate in the Hotel Booking System development.

3. System Architecture

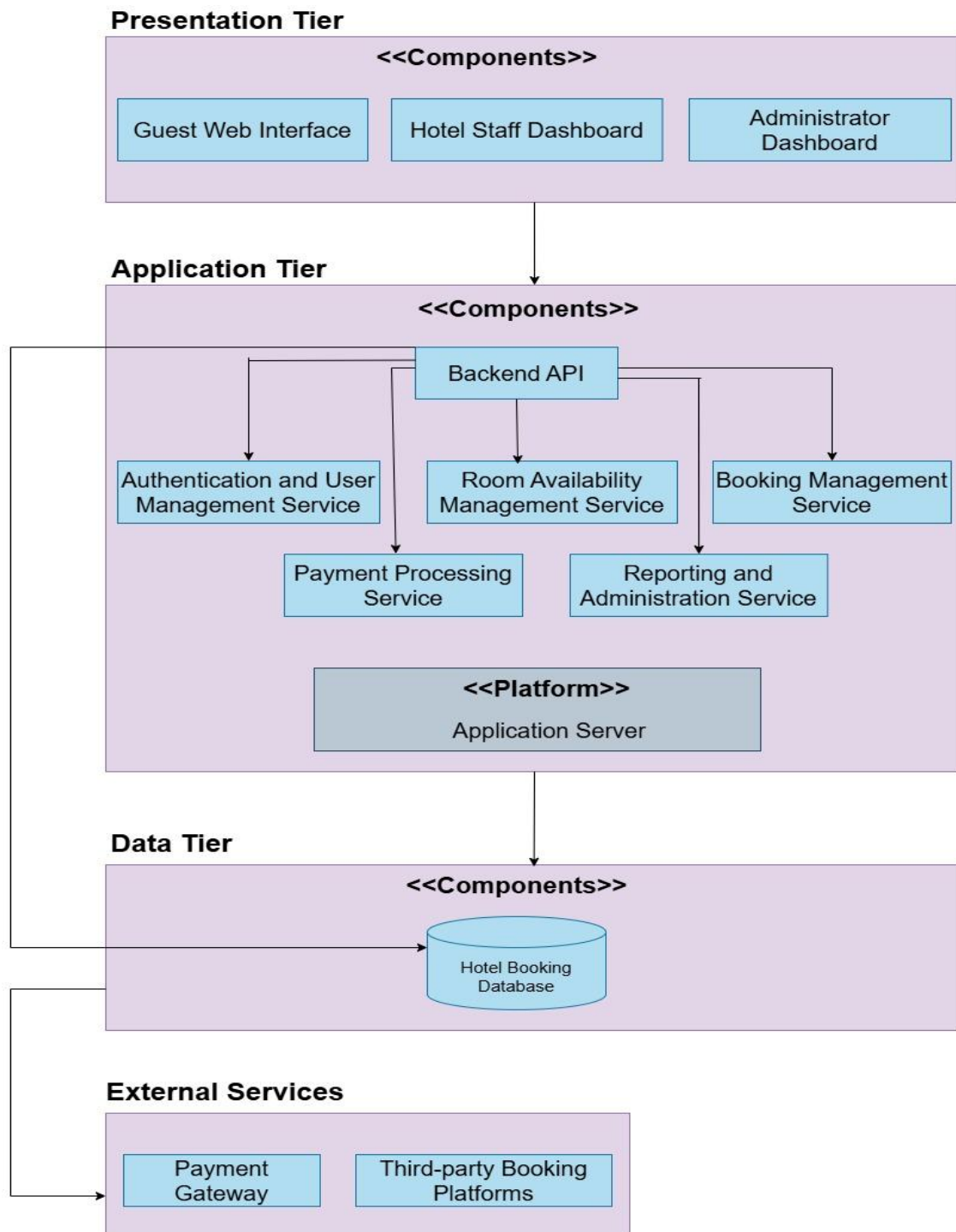


Figure 1: Architecture Diagram

3.1 Architectural Style

Based on functional requirements developed in Coursework 1, and the system context described by the onion diagram, the Hotel Booking System implements a three-tier architectural style with:

1. Presentation Tier
2. Application Tier
3. Data Tier

This architectural style supports well-defined separation of concerns, maintainability improvement, enhancement of security features, and scalability. It is peculiarly suitable for systems with several user roles, having online transactions with the public, and integrations with other external services such as payment gateways and third-party booking services.

3.2 Architectural Tiers, Platforms, and Components

❖ Presentation Tier

Components

- Guest Web Interface
- Hotel Staff Dashboard
- Administrator Dashboard

Platform

- Web browsers (desktop and mobile)

Responsibilities

- Receives user data (search, booking, payment)
- Shows hotel information, availability and bookings.
- Indicates various user roles as revealed in the use case diagram.

❖ **Application Tier**

Components

- Authentication and User Management Service
- Room Availability Management Service
- Booking Management Service
- Payment Processing Service
- Reporting and Administration Service

Platform

- Application Server

Responsibilities

- Deploys business logic based on CW1 use cases.
- Checks booking regulations and availability.
- Instructions payment processing.
- Makes administrative reports.

❖ **Data Tier**

Components

- Hotel Booking Database

Platform

- Relational Database Server

Responsibilities

- Stores user profiles, room, bookings and payments.
- Facilitates integrity and consistency of data.
- Transactional operations.

3.3 Connectors

Source	Destination	Connector Type
Presentation Tier	Application Tier	Web connector
Application Tier	Data Tire	Database Connector
Application Tier	Payment Gateway	External API Connector
Application Tier	Third-party Platforms	API Connector

Table 1: Connectors

3.4 Communication Types

- Client to Server – HTTPS (RESTful API)
- Application to Database – JDBC / ORM
- Payment Processing – Secure HTTPS API
- Third-party Integration – REST API

4. Testing of your Architectural Design

4.1 Realization of Functional Requirements (FRs)

The presentation, application, and data tiers working together in the proposed three tiers of architecture make the functional requirements of Coursework 1 come true.

FR1: Register/Login

Register/Login is carried out with the presentation layer where a user inserts the credentials and the authentication service in application layer that cross-checks the user information with the database. This makes system functionality secure and controlled.

FR2: Search for Hotels and Rooms

The search of the Hotels and Rooms is supported by the services of the room availability in the application tier which receives the search requests and returns the updated room data in the data tier.

FR3: Make Booking

Make Booking is achieved by booking management service that takes into consideration booking rules, checks the availability of rooms, and generates booking records that are put into the database.

FR4: Process Payment

Process Payment Process involves the payment processing service that has been integrated with an external payment gateway via RESTful APIs to make payments. A combination of these architectural elements facilitates in a well-organized and dependable way the fact that all the fundamental functional requirements are met.

4.2 Realization of Non-Functional Requirements (NFRs)

The non-functional requirements are met by the means of certain architectural decisions and communication means.

NFR1: Security

Security Customary role-based authentication and secure password management and encrypted HTTPS communication will be applied at all levels. The process of payments is transferred to a reputable third-party payment gateway, which minimizes the availability of sensitive financial information and maximizes the overall security of the system.

NFR2: Reliability

Reliability is achieved by centralization of data in the data tier and the application of transactional operations in the creation of bookings and processing of payment. The booking management service ensures successful updates on the database before booking confirmations are sent to the users and this ensures that there are no inconsistent or similar bookings. These design decisions will make the system secure and reliable in operation.

4.3 Realization of Technical Requirements (TRs)

Technical specifications of the system are met by using appropriate technologies, platforms, and connectors. The system is configured as a web application which allows one to search and book into the hotel using the normal web browsers. The application server provides the application layer which interacts with the data layer via database connectors (e.g. JDBC or ORM frameworks) to provide consistent data storage. External systems such as the payment gateway are integrated using RESTful APIs over HTTPS providing secure and standard communication. Such technical decisions are in line with industry best practices and make sure that the system is running in an efficient, secure, and with regard to the defined technical requirements.

5. Class Diagram and Justification

5.1 Class Structure

The class diagram contains the necessary classes that are necessary to aid the main functionality of the Hotel Booking System. User superclass defines the common authentication behavior and the subclasses include Guest and Staff which are different system roles. Domain entities like Hotel, Room, Booking, and payment are provided to facilitate the hotel search, booking of rooms, management of the booking, and payment. This framework is capable of supporting the functional needs that have been discovered in Coursework 1.

5.2 Identification of Attributes and Actions

Attributes and methods are part of each of the classes and assist in CRUD operations that are pertinent to the system. User account information is administered by the User class, and booking creation, editing, and cancellation are administered by the Booking class. Room class holds the room specific information and aids in checking availability and Hotel class holds information needed by search and display. The Payment class deals with payment information and handling of transactions. Such division of responsibility and duties guarantees proper data management and the existence of operational accountability.

5.3 Justification for Classes, Attributes, and Actions

The User class is a representation of authentication and profile-based information so that it can be reused between user roles. The Guest class deals with customer-facing activities like searching hotels, rooms and booking aligning with user-centered use cases. The Staff category assists in administration including how to handle the bookings and room availability. Accommodation data that is needed to search and verify availability are stored and operated through Hotel and Room classes. The Booking class serves as a key transactional point that connects the guests, rooms and payments with the Payment class that provides safe and reliable financial operations. Each of the classes and their attributes and actions is added depending on their relevance in the work of the system.

5.4 Correct UML Notation

The class diagram is based on the standard UML notation where the classes are depicted with three compartments namely: class name, attributes, and operations. Generalization arrows are used to model the relationships of inheritance amongst User, Guest and Staff with associations between the domain entities using the conventional association lines with proper multiplicity restrictions. The diagram is well defined, consistent and has not been overloaded with irrelevant complexity that may cause either inaccuracy or complications in understanding.

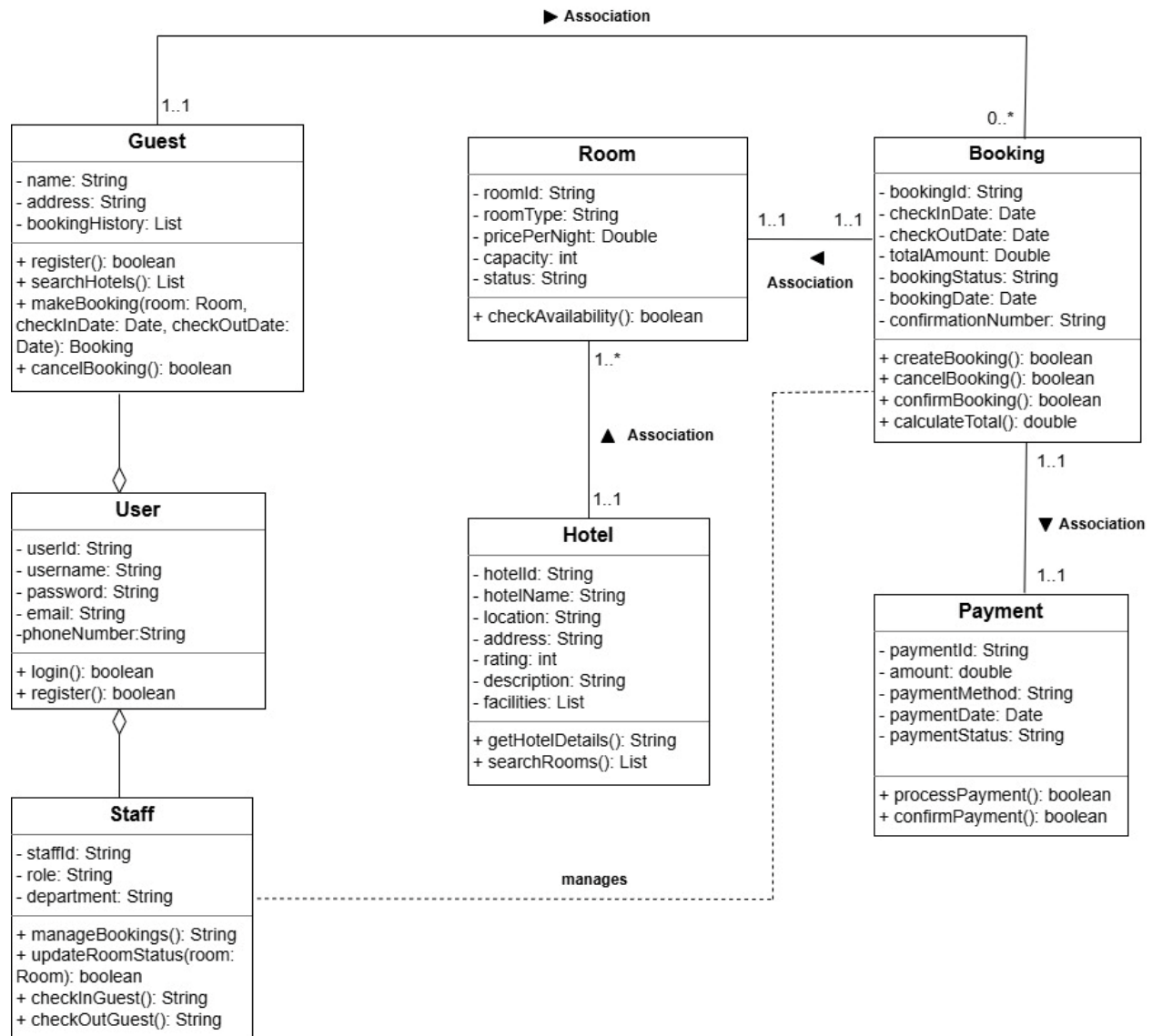


Figure 2: Class Diagram

6. Class Design Testing

6.1 Selected Core Functionality

The core functionality that is to be tested regarding the class design is the Make Booking and Process Pay. This is not a trivial functionality and is one of the main business processes of Hotel Booking System, and it includes numerous classes and interactions, such as the creation of booking, checking the availability of rooms, and payment.

6.2 UML Sequence Diagram Description

The UML sequence diagram will illustrate how the system objects interact in the event of a guest who makes a room booking and pays the booking. This process starts with a request to make a booking by the Guest using the system interface. The Booking class receives the request and authenticates the booking information and checks into room availability with the Room class. After the availability is checked, the Booking class computes the total cost and forms a booking record. The invocation of the Payment class is then taken to carry out the payment relating to the booking. After the confirmation payment has successfully been received, the booking status is updated and confirmation message is sent back to the guest. The sequence diagram adheres to the proper UML model, activation bars, and synchronous messages as well as return messages are present and sufficient to make it complete and accurate.

6.3 Required Classes and Their Roles

The order of sequence diagram includes Guest, Booking, Room and Payment classes, which are represented by the class diagram. The booking process starts with the Guest class in which the `makeBooking ()` operation is used to make the bookings. Room class facilitates the process by supporting the `check Availability` operation that makes sure that the room you have chosen is available during the dates requested. The Booking class that serves as the key coordinator of the functionality, with its attributes of `checkInDate`, `checkOutDate`, `bookingStatus`, `totalAmount`, manage the booking lifecycle. The Payment class uses the `processPayment ()` and `confirmPayment ()` operations in order to make the transaction and check that the payment status is properly documented. All these classes, attributes, as well as actions contribute to the efficient implementation of the booking and payment functionality.

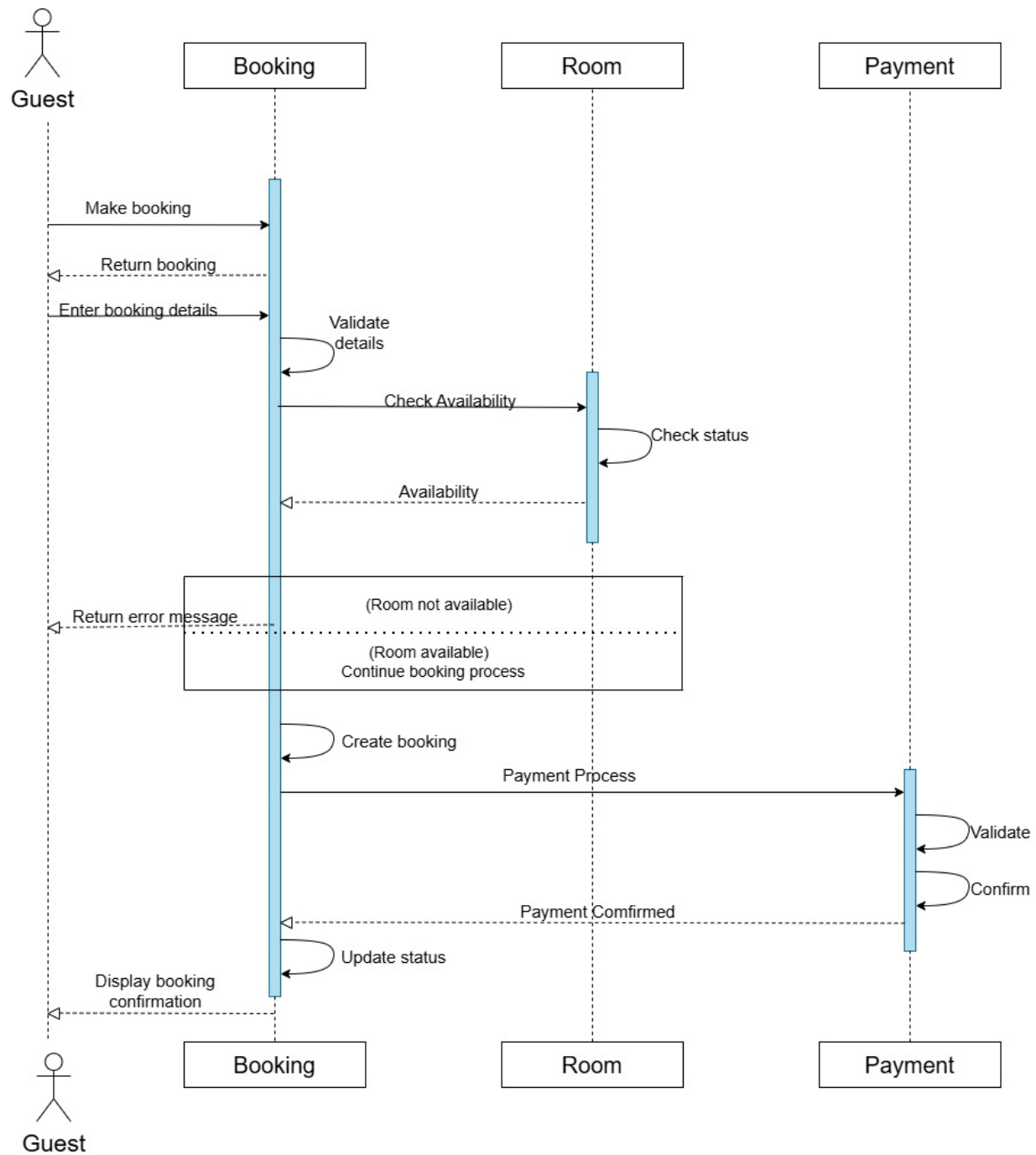


Figure 3: Sequence Diagram

7. Considerations of Modern Tools

In this section, a discussion will be made of the modern development, testing and implementation tools chosen in developing the Hotel Booking System explaining why they are suitable when it comes to the technical requirements and architectural design of the system itself.

7.1 Development and Testing Tools

- **Integrated Development Environment (IDE)**

The choice of IntelliJ IDEA as the main development environment is explained by the fact that it provides a high level of support to object-oriented programming, refactoring, debugging, and connection with the build and test tools. Its characteristics enhance quality of code and productivity of the developers in the system development.

- **Version Control System (VCS)**

Git, which is accompanied by GitHub is the version control. Git allows effective monitoring of the code changes, working collectively and undoing of the earlier versions. GitHub facilitates Agile development with issuance, forking, and pull requests under its features, which are in correspondence to incremental developmental strategy.

- **Dependency Management Tool**

Maven is utilized to handle the project's dependencies and configuration to build and maintain consistency within project's development environment while providing a mechanism to incorporate other external libraries like database drivers, security libraries, as well as testing libraries.

- **Frameworks and Libraries**

The backend development environment is aided by Spring Boot, which makes it easier to create RESTful services for handling bookings and payments as well as authentication. Spring Boot promotes modularity and also supports the layering concept that the system uses.

- **API and Testing Tools**

Postman is chosen for testing REST APIs provided by the application layer. It can be used to check the request and response validation, error handling, and performance testing. It uses the JUnit testing framework for unit testing business components including business validation and payment functionality.

7.2 Implementation Tools

- **Operating System**

The reason for using a Linux-based operating system is its maturity and good resource utilization. The capabilities it offers include high availability and confidentiality and integrity of data and other advanced features such as file permission security and process isolation.

- **Application and Web Servers**

Apache Tomcat acts as the application server where the back-end services are hosted. Apache Tomcat supports Java application services; it works well with RESTful APIs implemented in the system design.

- **Virtualization and Containerization**

Docker is used to create containers for system components like the application server and database. This ensures consistency, makes deployment simpler, and helps achieve scaling on various platforms.

- **Cloud-Based Platforms**

Cloud platforms such as AWS or Microsoft Azure are selected for hosting the infrastructure of the system. Cloud platforms ensure scalability, load balancing, auto-backup, and disaster recovery, which include the non-functional requirements of the system.

- **External APIs**

The payment gateway API is developed according to PCI-DSS standards and incorporated to ensure secure online payment services. The external API will ensure secure payment operations with negligible risks involved in handling payment information.

8. Considerations of the Software Engineering principles in your design

This part is a reflection of how the main principles of software engineering were applied in the design of the Hotel Booking System. The discussion is about the realization of the selected principles via the system architecture and design of the classes rather than giving the abstract definition.

8.1 Separation of Concerns

- The concept of the separation of concerns is achieved by adopting a three-tier architectural design. Presentation tier only involves interaction with the user, through registration, searching and booking of rooms. The business logic is stored in the application layer and it comprises the booking management, payment processing, and authentication services. Data persistence and integrity are taken care of by the data tier. This distinct separation means that user interfaces or business rule changes can be done without considering database operations and enhances maintainability as well as simplifying systems.

8.2 Modularity

- The way in which modularity is applied is by breaking down the application tier and creating independent services, including the booking management service, room availability service, payment processing service, and authentication service. The modules clearly define their responsibility and interface with other modules in clearly defined interfaces. Indicatively, the payment processing service can be adjusted or substituted without affecting the logic in controlling the booking. This modular architecture improves testability, ease of debugging and allows one to develop in parallel.

8.3 Abstraction

- One of the methods used to accomplish abstraction is concealing the implementation details by service interfaces and class definitions. The payment processing service isolates communications with third party payment gateways, making booking logic no longer dependent on any particular payment provider. Equally, application logic is not exposed to database-specific details through the use of abstract data access mechanisms, including ORM frameworks. This minimizes the complexity of the system and enables internal realizations to be modified without impacting the high-level components.

8.4 Anticipation of Change

- The design is also future-friendly as it is capable of supporting architectural and class extensibility. The service-based application layer can be reconfigured to add new features like new payment options, loyalty schemes or third-party booking solutions with only minor modifications to the existing modules. Extension is also supported by the class diagram enabling the introduction of new user roles or types of rooms without reorganizing classes. This preventive design method minimizes future maintenance work.

8.5 Incremental Development

- The Agile methodology is used to achieve incremental development, in which the system will be built and developed through iterative development. The basic features included user registration, room search, and room booking are deployed initially and later on some other features, including payment integration and report generation are added. This incremental method enables the on-going validation of design choices, the early identification of problems, and the progressive increase of system number of features without losing track with the requirements of the stakeholders.

References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson Education, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [3] UML.org, "Unified Modeling Language (UML) Specification," Object Management Group, 2023. [Online]. Available: <https://www.omg.org/spec/UML/>
- [4] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [5] Oracle, "Java Platform, Standard Edition Documentation," Oracle Corporation, 2024. [Online]. Available: <https://docs.oracle.com/javase/>
- [6] Spring, "Spring Boot Reference Documentation," VMware, Inc., 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [7] Amazon Web Services, "AWS Cloud Architecture Best Practices," AWS, 2024. [Online]. Available: <https://aws.amazon.com/architecture/>
- [8] Postman, "API Development and Testing," Postman Inc., 2024. [Online]. Available: <https://learning.postman.com/>

Appendix

Appendix ID	Title	Description
Appendix A	System Architecture Diagram	Shows the three-level architecture of the Hotel Booking System, as to presentation, application and data layer, and external system integration as the payment gateway.
Appendix B	UML Class Diagram	Shows the major class structure of Hotel Booking System along with User, Guest, Staff, Hotel, Room, Booking, and Payment classes, their attributes, operations, inheritance, associations, and multiplicity.
Appendix C	UML Sequence Diagram	Reveals how the interaction between the objects of the system takes place when the functionality of Making a Booking and Processing a Payment is executed, confirming the design and behavior of the classes.

Appendix D	Tools and Technologies Summary	Gives an overview of the development, testing, and implementation tools that were chosen to work on the project and they include programming languages, frameworks, version control systems, and deployment platforms.
Appendix E	Assumptions and Design Constraints	Identifies the assumptions and limitations that have been taken into account when designing the system, including: system access, system transactions, scalability expectations, and deployment environment.